

并行程序设计 with 算法实验

实验四：基于 OpenMP 的并行计算

姓名：元朗曦 学号：23336294

2026 年 5 月 6 日

1 实验目的

本实验的目标是使用 OpenMP 实现基于共享内存的并行计算，包括矩阵乘法和数组求和两个典型算法。通过本实验希望达到以下目的：

1. 掌握 OpenMP 的基本编程模型和指令集，特别是 `#pragma omp parallel for` 的使用。
2. 理解并行循环中的工作分配、负载均衡与 `schedule` 策略的影响。
3. 学习规约（reduction）操作在数据并行中的应用，避免竞争条件。
4. 通过性能测试验证线程数增加对程序运行时间的影响，分析加速比和并行效率。
5. 对比 OpenMP 与传统线程库（如 POSIX 线程）的编程便利性。

2 实验原理

2.1 OpenMP 并行编程模型

OpenMP（Open Multi-Processing）是一个基于共享内存的并行编程框架，通过编译指导语句（pragma）和运行时库函数，提供高层次的并行抽象。相比 POSIX 线程库（pthread），OpenMP 提供了更简洁的 API，程序员无需手动创建和销毁线程，编译器会自动生成并行代码。

OpenMP 的核心编程模型基于“fork-join”范式：主线程执行串行代码，在遇到并行指令时创建工作线程，所有线程并行执行指定的代码块，然后在隐含屏障（implicit barrier）处同步，主线程继续执行后续串行代码。

2.2 并行循环与工作分配

在数据并行中，最常见的模式是使用 `#pragma omp parallel for` 将循环迭代分配给多个线程。调度策略通过 `schedule()` 子句指定：

1. **静态调度** (`schedule(static)`)：将迭代均匀分配给各线程，无运行时开销，但若迭代工作量不均则负载不均衡。
2. **动态调度** (`schedule(dynamic)`)：运行时将迭代动态分配，负载均衡好但同步开销大。
3. **自适应调度** (`schedule(guided)`)：介于两者之间，初期大块分配，后期逐步减小块大小。

对于矩阵乘法中行与行之间独立的外层循环，使用静态调度足以获得良好的负载均衡。

2.3 规约操作

数组求和等操作涉及多个线程的累积。直接在并行区域中共享变量会产生竞争条件。OpenMP 提供 `reduction(op:var)` 子句来安全地累积结果：编译器为每个线程创建临时副本，各线程独立计算，最后通过指定的操作（如 +）合并结果。

2.4 浮点性能指标

浮点计算性能用 GFLOPS（十亿浮点运算次数/秒）表示：

$$\text{GFLOPS} = \frac{\text{浮点运算次数}}{t \times 10^9} \quad (1)$$

对于矩阵乘法 $C = A \times B$ ，其中 $A \in \mathbb{R}^{m \times n}$ ， $B \in \mathbb{R}^{n \times k}$ ，浮点运算次数约为 $2mnk$ （ n 次乘-加操作，每个迭代）。

加速比定义为：

$$\text{Speedup} = \frac{t_{\text{serial}}}{t_{\text{parallel}}} \quad (2)$$

理想情况下，用 p 个线程运行程序的加速比应接近 p （线性加速），但实际受到通信开销、负载不均衡、Amdahl's Law 等限制。

3 实验内容

3.1 程序设计

本实验在 `lab/04/src` 中实现两个核心程序：

3.1.1 矩阵乘法 (matmul_openmp)

- 功能：计算 $C = A \times B$ ，其中 $A \in \mathbb{R}^{m \times n}$ ， $B \in \mathbb{R}^{n \times k}$ 。
- 输入：命令行参数 <m> <n> <k> <threads> [--verify]。
- 输出：矩阵形状、线程数、运行时间、GFLOPS、可选的校验结果。
- 并行策略：使用 `#pragma omp parallel for schedule(static)` 并行化外层行迭代。
- 校验：与串行版本比较，检查数值精度（最大绝对误差 $< 10^{-8}$ ）。

3.1.2 数组求和 (array_sum_openmp)

- 功能：计算数组元素和 $\sum_{i=0}^{N-1} a[i]$ 。
- 输入：命令行参数 <size> <threads> [--verify]。
- 输出：数组规模、线程数、运行时间、吞吐量 (Gelem/sec)、结果、可选的校验。
- 并行策略：使用 `#pragma omp parallel for reduction(+:sum)` 安全地累积和。
- 校验：与串行版本比较，检查精度（误差 $< 10^{-6}$ ）。

3.2 构建与运行

使用 CMake 作为构建工具：

```

1 # 配置和构建
2 mkdir -p lab/04/build
3 cd lab/04/build
4 cmake ..
5 cmake --build . -j
6
7 # 运行矩阵乘法 (512x512x512, 4线程, 含验证)
8 ./src/matmul_openmp 512 512 512 4 --verify
9
10 # 运行数组求和 (1000万元素, 4线程, 含验证)
11 ./src/array_sum_openmp 10000000 4 --verify
12
13 # 性能测试：不同线程数下的加速比
14 for t in 1 2 4 8; do
15     ./src/matmul_openmp 1024 1024 1024 $t
16 done

```

4 实验结果

4.1 正确性验证

两个程序都通过了数值校验测试，验证无误。

4.1.1 矩阵乘法校验 ($512 \times 512 \times 512$, 4 线程)

```
1 Task: matrix_multiplication_omp
2 Shape: A(512x512), B(512x512)
3 Threads: 4
4 Time (sec): 0.162897
5 GFLOPS: 1.648
6 Verify: PASS, max_abs_diff=0.0000000000
```

所有元素与串行版本完全相同（误差在浮点精度范围内）。

4.1.2 数组求和校验 (1000 万元素, 4 线程)

```
1 Task: array_sum_omp
2 Size: 10000000
3 Threads: 4
4 Time (sec): 0.006282
5 Throughput (Gelem/sec): 1.591899
6 Result: 1789.129084
7 Verify: PASS, serial_result=1789.1290836077, diff=0.0000000001
```

规约操作正确，避免了竞争条件。

4.2 性能分析：矩阵乘法加速比

测试环境： $1024 \times 1024 \times 1024$ 矩阵乘法，不同线程数下的性能表现。

表 1: OpenMP 矩阵乘法性能扩展性

线程数	时间 (秒)	GFLOPS	加速比	效率 (%)
1	4.950	0.434	1.00	100.0
2	2.518	0.853	1.97	98.3
4	1.298	1.655	3.81	95.3
8	0.860	2.496	5.75	71.9

4.3 结论与分析

4.3.1 线性加速的实现

从 1 线程到 4 线程，加速比接近线性 ($1\times$ 、 $2\times$ 、 $4\times$)，平行效率分别为 100%、98.3%、95.3%，说明在这个范围内，OpenMP 的开销很小，并行收益显著。

4.3.2 8 线程处的性能下降

8 线程时加速比为 $5.75\times$ （而非理想的 $8\times$ ），平行效率降至 71.9%。这是因为：

1. 当前测试环境可能只有 4 个物理核心；过度订阅（oversubscription）会增加上下文切换开销。
2. OpenMP 线程创建、销毁及同步的开销在线程数增多时变得相对更显著。
3. 缓存行竞争（cache line contention）在高线程数下加剧。

4.3.3 与 pthread 的对比

相比 lab/03 的 pthread 实现（matmul_pthreads 最多 16 线程），OpenMP 提供了以下优势：

- **编程简洁性**：无需手动管理线程生命周期，只需一条 pragma 指令。
- **灵活的调度策略**：通过 schedule 子句可轻易调整工作分配方式。
- **规约支持**：内置 reduction 机制，避免竞争条件和锁竞争。
- **可移植性**：基本 OpenMP 功能在各种编译器上都有支持。

同时，OpenMP 对共享内存架构的优化比消息传递（MPI）更直接，适合单机多核、SMP 系统。

4.3.4 实验收获

1. 掌握了 OpenMP 的核心用法，能够快速将串行程序并行化。
2. 理解了共享内存编程中规约、同步、负载均衡的关键概念。
3. 通过实测加速比，验证了理论上的加速上界与实际性能的差异。
4. 认识到无论多核还是众核系统，仍需关注缓存、负载均衡、同步开销等细节。

总结

本实验成功演示了 OpenMP 在矩阵乘法和数组求和中的应用。在线程数不超过物理核心数时，程序表现出接近线性的加速比；超出物理核心数后，加速比增长放缓。OpenMP 的高级抽象大幅降低了并行编程难度，同时保持了足够的性能，是现代共享内存并行计算的主流选择。